

Επίλυση Προβλήματος Αναζήτησης σε Ψηφιακό Καταχωρητή.



Συγκριτική Μελέτη Αλγορίθμων: BFS, DFS, Best-First και A*.

ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ

Τμήμα Εφαρμοσμένης Πληροφορικής

Μάθημα: Τεχνητή Νοημοσύνη

Ονοματεπώνυμο Φοιτητή: Μάριος-Βασίλειος Μανθέλας

Αριθμός Μητρώου: ics24169

Email: ics24169@uom.edu.gr

Υπεύθυνος Καθηγητής: Γιάννης Ρεφανίδης

Περιεχόμενα

1. ΕΙΣΑΓΩΓΗ	2
2. Περιγραφή Κώδικα	2
2.1. Γενική Περιγραφή Υλοποίησης	2
2.2. Αναπαράσταση Καταστάσεων και Κόμβων	2
2.3. Περιγραφή Ευρετικής Συνάρτησης	3
2.4. Χρησιμοποιούμενες Βιβλιοθήκες	3
3. Πειραματική Διαδικασία	3
3.1. Γενική Περιγραφή Πειραματικής Διαδικασίας	3
3.2. Απαρίθμηση Προβλημάτων	4
3.3. Στιγμιότυπα Εκτέλεσης	4
4. Αναλυτικά Αποτελέσματα και Συγκριτικός Σχολιασμός Αλγορίθμων	6
4.1. Πίνακας Αναλυτικών Πειραματικών Δεδομένων	6
4.2. Σύγκριση Χρόνου Εκτέλεσης	6
4.3. Σύγκριση Μήκους Λύσης	7
4.4. Σύγκριση Κόστους Λύσης	7
4.5. Τελικά Συμπεράσματα και Συνολική Αξιολόγηση	8

Εικόνες

Εικόνα 1: Κατηγορία Α, Σενάριο(100->1)	5
Εικόνα 2: Κατηγορία Β, Σενάριο(5->160)	5
Εικόνα 3: Κατηγορία Γ Σενάριο(10->500)	5
Εικόνα 4: Κατηγορία Δ, Σενάριο(10->100000)	6

Πίνακες

Πίνακας 1: Αναλυτικά Πειραματικά Δεδομένα(Βήματα / Κόστος / Χρόνος σε sec)	6
Πίνακας 2: Διάγραμμα Χρόνου Εκτέλεσης	7
Πίνακας 3: Διάγραμμα Μήκους Λύσης	7
Πίνακας 4: Σύγκριση Κόστους Λύσης	8

1. ΕΙΣΑΓΩΓΗ

Η παρούσα εργασία αφορά στην επίλυση ενός προβλήματος αναζήτησης σε χώρο καταστάσεων, με αντικείμενο έναν ψηφιακό καταχωρωτή τιμών(χ). Ο καταχωρητής σύμφωνα με την εκφώνηση επιτρέπει την εκτέλεση έξι συγκεκριμένων απλών μαθηματικών πράξεων: αύξηση ($\chi+1$), μείωση ($\chi-1$), διπλασιασμό ($\chi*2$), υποδιπλασιασμό ($\chi/2$), τετραγωνισμό (χ^2) και τετραγωνική ρίζα ($\sqrt{\chi}$). Κάθε πράξη από τις παραπάνω έχει ένα μεταβαλλόμενο κόστος το οποίο εξαρτάται από την τρέχουσα τιμή του καταχωρητή.

Ο σκοπός της άσκησης είναι η εύρεση ενός μονοπατιού πράξεων που οδηγεί από μια αρχική τιμή σε μια τιμή-στόχο. Για την επίλυση αυτού του προβλήματος υλοποιήθηκαν και συγκρίθηκαν τέσσερις διαφορετικοί αλγόριθμοι αναζήτησης:

- **Αναζήτηση σε πλάτος(Breadth first search).**
- **Αναζήτηση σε βάθος(Depth first search).**
- **Αναζήτηση πρώτα στο καλύτερο(Best first search/Greedy).**
- **Αναζήτηση A*(Αλφα Άστρο)(A-star).**

Κρίσιμο σημείο της εργασίας αποτέλεσε η εύρεση και ο σχεδιασμός μιας **παραδεκτής (admissible)** ευρετικής συνάρτησης, η οποία επιτρέπει στον αλγόριθμο A* να εγγυάται τη βέλτιστη λύση, καθώς και η πειραματική ανάλυση της συμπεριφοράς των αλγορίθμων σε σενάρια διαφορετικής δυσκολίας και βάθους.

2. Περιγραφή Κώδικα

2.1. Γενική Περιγραφή Υλοποίησης

Ο κώδικας υλοποιεί ένα πρόβλημα αναζήτησης κατάστασης στο οποίο κάθε κατάσταση αντιστοιχεί σε έναν ακέραιο αριθμό. Στόχος του προγράμματος είναι η μετάβαση από μία αρχική τιμή σε μία τελική τιμή, εφαρμόζοντας ένα σύνολο επιτρεπτών αριθμητικών πράξεων με διαφορετικά δυναμικά κόστη.

Στον κώδικα υλοποιήσαμε τέσσερις αλγορίθμους αναζήτησης:

- **Αναζήτηση σε πλάτος(Breadth first search).**
- **Αναζήτηση σε βάθος(Depth first search).**
- **Αναζήτηση πρώτα στο καλύτερο(Best first search/Greedy).**
- **Αναζήτηση A*(Αλφα Άστρο)(A-star).**

Οι παραπάνω αλγόριθμοι χρησιμοποιούν κοινό κορμό υλοποίησης και διαφοροποιούνται αποκλειστικά από τον τρόπο που αντιμετωπίζουν την εισαγωγή και εξαγωγή κόμβων από τη δυναμική λίστα frontier.

2.2.Αναπαράσταση Καταστάσεων και Κόμβων

Δημιουργήσαμε μια δομή(struct) η οποία αναπαριστά τον κάθε κόμβο του δέντρου αναζήτησης και περιέχει τις εξής τιμές:

- **state:** η τρέχουσα αριθμητική τιμή (κατάσταση),
- **g:** το συσσωρευμένο κόστος από την αρχική κατάσταση,
- **h:** η τιμή της ευρετικής συνάρτησης,
- **f:** η προτεραιότητα του κόμβου (h ή $g+h$),
- ***parent:** δείκτης στο γονικό κόμβο,
- **action:** η πράξη που οδήγησε σε αυτή την κατάσταση.

2.3. Περιγραφή Ευρετικής Συνάρτησης

Για την υλοποίηση των αλγορίθμων Best-First και A* δημιουργήσαμε τη δική μας παραδεκτή ευρετική συνάρτηση με σκοπό την εύρεση του μονοπατιού στο δέντρο καταστάσεων με τον μικρότερο δυνατό συνδυασμό επεκτεινόμενων κόμβων και κόστους. Το πρόβλημα που μελετάμε αφορά στη μετατροπή μιας αρχικής αριθμητικής τιμής σε μία τελική τιμή στόχο, μέσω ενός συνόλου επιτρεπτών αριθμητικών πράξεων με δυναμικά κόστη. Στο πλαίσιο αυτό, η απλή διαφορά μεταξύ τρέχουσας κατάστασης και στόχου δεν αποτελεί αξιόπιστη εκτίμηση κόστους, καθώς πράξεις όπως ο διπλασιασμός, ο υποδιπλασιασμός, ο τετραγωνισμός και η τετραγωνική ρίζα μπορούν να μεταβάλλουν δραστικά το μέγεθος της τιμής σε ένα μόνο βήμα.

Έχοντας λάβει υπόψη μας τα παραπάνω, η ευρετική συνάρτηση σχεδιάστηκε ώστε να εκτιμά όχι την ακριβή απόσταση από τον στόχο, αλλά την τάξη μεγέθους της διαφοράς αγνοώντας τα πραγματικά κόστη των πράξεων. Συγκεκριμένα, η ευρετική συνάρτηση που χρησιμοποιήσαμε ορίζεται ως:

$$h(x) = \lfloor \log_2(|x - goal|) \rfloor$$

με $h(goal) = 0$, όπου x είναι η τρέχουσα κατάσταση και $goal$ η τελική τιμή.

Η παραπάνω ευρετική συνάρτηση είναι παραδεκτή καθώς δεν υπερεκτιμά ποτέ το πραγματικό ελάχιστο κόστος της λύσης.

Απόδειξη:

Έστω $D = |x - goal|$. Για να καταφέρουμε να μειώσουμε αυτήν την απόσταση στο μηδέν, ακόμη και στην πλέον αισιόδοξη περίπτωση, απαιτούνται τουλάχιστον $\lfloor \log_2(D) \rfloor$ μεταβολές τάξης μεγέθους. Επιπλέον κάθε επιτρεπτή πράξη έχει θετικό κόστος το οποίο μας βοηθάει να συμπεράνουμε πως το πραγματικό ελάχιστο κόστος $h^*(x)$ ικανοποιεί:

$$h^*(x) \geq \lfloor \log_2(D) \rfloor = h(x)$$

Άρα ισχύει $h(x) \leq h^*(x)$ για κάθε κατάσταση x , και η ευρετική συνάρτηση είναι παραδεκτή.

Η συγκεκριμένη ευρετική συνάρτηση μας βοήθησε να μειώσουμε δραστικά το πλήθος των επεκτάσεων και το συνολικό κόστος λύσης συγκρίνοντάς τα με το πλήθος των επεκτάσεων και το συνολικό κόστος λύσης των απληροφόρητων αλγορίθμων αναζήτησης.

2.4. Χρησιμοποιούμενες Βιβλιοθήκες

Για την υλοποίηση των αλγορίθμων αναζήτησης σε γλώσσα C χρησιμοποιήσαμε τις παρακάτω πρότυπες βιβλιοθήκες:

```
#include <time.h>
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

3. Πειραματική Διαδικασία

3.1. Γενική Περιγραφή Πειραματικής Διαδικασίας

Για την πειραματική αξιολόγηση των αλγορίθμων επιλέξαμε αρχικά ένα σύνολο αριθμών αρχικής και τελικής κατάστασης, στην συνέχεια εκτελέσαμε το πρόγραμμα τέσσερις φορές για το κάθε ζεύγος χρησιμοποιώντας

κάθε φορά διαφορετικό αλγόριθμο αναζήτησης, έτσι ώστε τα αποτελέσματα να είναι άμεσα συγκρίσιμα μεταξύ των αλγορίθμων.

3.2.Απαρίθμηση Προβλημάτων

Για να καλύψουμε όλο το φάσμα λειτουργίας των αλγορίθμων, επιλέξαμε να τον εκτελέσουμε σε συνολικά 21 διαφορετικά ζεύγη τιμών τα οποία εκπροσωπούν τις εξής κατηγορίες ελέγχων:

A. Βασικοί Έλεγχοι-Ορθότητας

Στόχος είναι ο έλεγχος βασικών λειτουργιών των αλγορίθμων σε μικρά, διαχειρίσιμα διαστήματα αναζήτησης, καθώς και ο έλεγχος της ικανότητας του προγράμματος να εκτελεί σωστά όλους τους τελεστές.

Ζεύγη Τιμών:

- I. 1->10
- II. 100->1
- III. 3->10
- IV. 2->1
- V. 64->7
- VI. 10000->2

B. Έλεγχος Βελτιστοποίησης Κόστους

Στόχος είναι η σύγκριση των BFS με τον Best-First και A* δημιουργώντας παγίδες κόστους, δηλαδή προβλήματα όπου το μονοπάτι με τα λιγότερα βήματα δεν είναι το φθηνότερο:

- I. 4->256
- II. 15->100
- III. 5->160

C. Σενάρια Κλιμακούμενης Δυσκολίας

Στόχος μας είναι να δείξουμε πως η αύξηση του μεγέθους του προβλήματος οδηγεί σε αύξηση του χρόνου εκτέλεσης:

- IV. 10->500
- V. 50->1000
- VI. 45->10000
- VII. 2->20000
- VIII. 10->50000

D. Σενάρια Με Κοινή Αφετηρία

Στόχος μας είναι να δημιουργήσουμε αποτελέσματα τα οποία να δείχνουν την επίδραση της δραστηκής αύξησης μεγέθους λύσης του προβλήματος σε αντιστοιχία με την αύξηση του μεγέθους του προβλήματος και να μπορούν ταυτόχρονα να χρησιμοποιηθούν για δημιουργία διαγραμμάτων απόδοσης:

- IX. 10->2000
- X. 10->10000
- XI. 10->30000
- XII. 10->60000
- XIII. 10->100000

3.3.Στιγμιότυπα Εκτέλεσης

Παρακάτω παρουσιάζουμε μερικά στιγμιότυπα εκτέλεσης, ένα από κάθε κατηγορία ελέγχου:

Εικόνα 1: Κατηγορία Α, Σενάριο(100->1)

```
register.exe breadth 100 1 out2_breadth.txt
Solving 100 using breadth...
Solution found! (4 steps)(33 cost)
Time spent: 0.000000 secs

register.exe depth 100 1 out2_depth.txt
Solving 100 using depth...
Solution found! (99 steps)(198 cost)
Time spent: 0.000000 secs

register.exe best 100 1 out2_best.txt
Solving 100 using best...
Solution found! (4 steps)(33 cost)
Time spent: 0.000000 secs

register.exe astar 100 1 out2_astar.txt
Solving 100 using astar...
Solution found! (4 steps)(31 cost)
Time spent: 0.000000 secs
```

Εικόνα 2: Κατηγορία Β, Σενάριο(5->160)

```
register.exe breadth 5 160 out3_breadth.txt
Solving 5 using breadth...
Solution found! (5 steps)(83 cost)
Time spent: 0.000000 secs

register.exe depth 5 160 out3_depth.txt
Solving 5 using depth...
Solution found! (157 steps)(315 cost)
Time spent: 0.000000 secs

register.exe best 5 160 out3_best.txt
Solving 5 using best...
Solution found! (41 steps)(170 cost)
Time spent: 0.000000 secs

register.exe astar 5 160 out3_astar.txt
Solving 5 using astar...
Solution found! (13 steps)(66 cost)
Time spent: 0.000000 secs
```

Εικόνα 3: Κατηγορία Γ Σενάριο(10->500)

```
register.exe breadth 10 500 out9_breadth.txt
Solving 10 using breadth...
Solution found! (8 steps)(229 cost)
Time spent: 0.003000 secs

register.exe depth 10 500 out9_depth.txt
Solving 10 using depth...
Solution found! (453 steps)(1363 cost)
Time spent: 0.000000 secs

register.exe best 10 500 out9_best.txt
Solving 10 using best...
Solution found! (102 steps)(302 cost)
Time spent: 0.000000 secs

register.exe astar 10 500 out9_astar.txt
Solving 10 using astar...
Solution found! (19 steps)(158 cost)
Time spent: 0.000000 secs
```

Εικόνα 4: Κατηγορία Δ, Σενάριο (10->100000)

```
register.exe breadth 10 100000 de_breadth.txt
Solving 10 using breadth...
Solution found! (16 steps)(49998 cost)
Time spent: 0.259000 secs

register.exe depth 10 100000 de.txt
Solving 10 using depth...
Timeout
No solution found.

register.exe best 10 100000 de_best.txt
Solving 10 using best...
Solution found! (18407 steps)(67460 cost)
Time spent: 3.391000 secs

register.exe astar 10 100000 de_astar.txt
Solving 10 using astar...
Solution found! (156 steps)(25276 cost)
Time spent: 32.092999 secs
```

4. Αναλυτικά Αποτελέσματα και Συγκριτικός Σχολιασμός Αλγορίθμων

4.1. Πίνακας Αναλυτικών Πειραματικών Δεδομένων

Πίνακας 1: Αναλυτικά Πειραματικά Δεδομένα (Βήματα / Κόστος / Χρόνος σε sec)

Σενάριο	BFS	DFS	Best	A*
1.(1->10)	4 / 10 / 0.000	9 / 18 / 0.000	5 / 11 / 0.000	4 / 9 / 0.000
2.(100->1)	4 / 33 / 0.000	99 / 198 / 0.000	4 / 33 / 0.000	4 / 31 / 0.000
3.(5->160)	5 / 83 / 0.000	157 / 315 / 0.000	41 / 170 / 0.000	13 / 66 / 0.000
4.(64->7)	2 / 17 / 0.000	57 / 114 / 0.000	2 / 17 / 0.000	2 / 17 / 0.000
5.(4->256)	2 / 65 / 0.000	378 / 1059 / 0.000	2 / 65 / 0.000	2 / 65 / 0.001
6.(15->100)	4 / 45 / 0.000	96 / 194 / 0.000	14 / 136 / 0.000	6 / 34 / 0.000
7.(3->10)	2 / 5 / 0.000	8 / 16 / 0.000	2 / 5 / 0.000	2 / 5 / 0.000
8.(2->1)	1 / 2 / 0.000	1 / 2 / 0.000	1 / 2 / 0.000	1 / 2 / 0.000
9.(10->500)	8 / 229 / 0.003	453 / 1363 / 0.000	102 / 302 / 0.000	19 / 158 / 0.000
10.(50->1000)	10 / 509 / 0.016	877 / 2644 / 0.000	253 / 979 / 0.000	33 / 325 / 0.004
11.(45->10000)	5 / 2522 / 0.000	8550 / 26325 / 0.322	3479 / 13175 / 0.028	7 / 2512 / 0.097
12.(10000->2)	4 / 2507 / 0.000	9998 / 19996 / 0.501	4 / 2507 / 0.000	5 / 2507 / 0.026
15.(2->20000)	6 / 7509 / 0.000	19998 / 39996 / 2.266	2498 / 13467 / 0.000	127 / 5223 / 0.394
16.(10->50000)	15 / 24997 / 0.075	72873 / 207149 / 50.976	9210 / 28665 / 0.469	184 / 12907 / 3.584
17.(10->2000)	10 / 981 / 0.016	1833 / 5559 / 0.017	510 / 1652 / 0.001	30 / 569 / 0.008
18.(10->10000)	2 / 2500 / 0.000	11497 / 33714 / 0.560	3963 / 14281 / 0.029	2 / 2500 / 0.094
19.(10->30000)	14 / 56315 / 0.074	31433 / 93551 / 6.231	10818 / 31881 / 0.125	80 / 7636 / 1.066
20.(10->60000)	15 / 71316 / 0.104	62873 / 187149 / 33.748	19010 / 67360 / 0.598	41 / 15088 / 5.067
21.(10->100000)	16 / 49998 / 0.126	--	18407 / 67460 / 2.129	156 / 25276 / 21.095

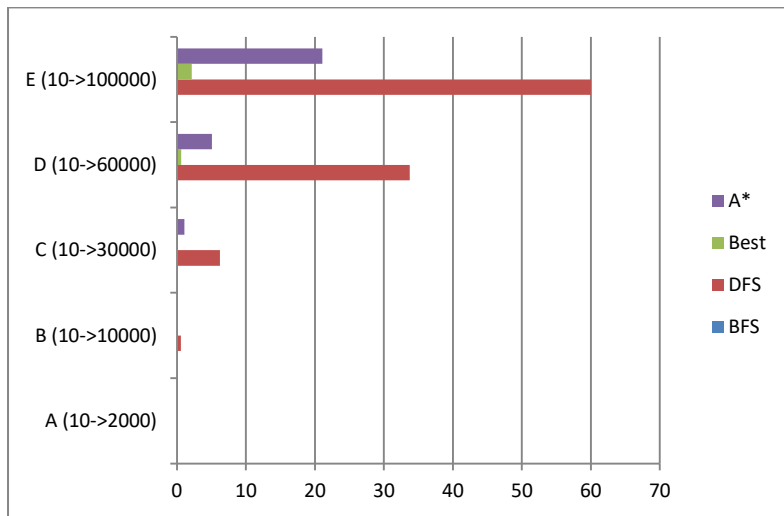
4.2. Σύγκριση Χρόνου Εκτέλεσης

Μελετώντας τα δεδομένα του παραπάνω πίνακα (Πίνακας 1) εξάγουμε το συμπέρασμα ότι στα μικρού μεγέθους προβλήματα ο χρόνος είναι πάντα αμελητέος (<0.000). Ωστόσο, στα μεγάλα σενάρια, όπως τα 10, 11, 12, προκύπτουν σημαντικές διαφορές:

- Ο **BFS** διατηρεί χαμηλό χρόνο σε όλες τις περιπτώσεις με το τίμημα του υψηλού κόστους.
- Ο **DFS** παρουσιάζει μεγάλη αύξηση χρόνου έως και 51 δευτερόλεπτα, ενώ τερματίζει μια φορά λόγω υπέρβασης χρονικού ορίου, η οποία προέκυψε από εκτεταμένη αναζήτηση του χώρου καταστάσεων, γεγονός που επιβεβαιώνει τη θεωρητική εκθετική πολυπλοκότητα.
- Ο **Best-First** είναι προφανώς ταχύτερος από τον **DFS** αλλά παραμένει μη αποδοτικός σε πολύ μεγάλα προβλήματα.

- Ο **A*** επιτυγχάνει σημαντική διαφορά χρόνου σε σχέση με τους **DFS** και **Best-First** ακόμη και σε πολύ μεγάλους χώρους αναζήτησης.

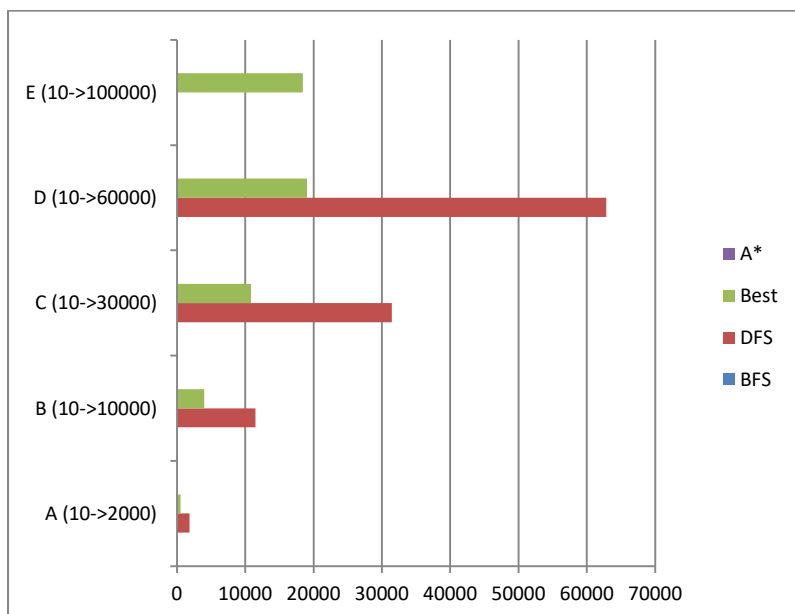
Πίνακας 2: Διάγραμμα Χρόνου Εκτέλεσης



4.3. Σύγκριση Μήκους Λύσης

- Ο **BFS** βρίσκει λύσεις με ελάχιστο αριθμό βημάτων.
- Ο **DFS** οδηγεί στις λύσεις με τον μεγαλύτερο αριθμό βημάτων.
- Ο **Best-First** εμφανίζει ενδιάμεση συμπεριφορά, συχνά παγιδεύομενος σε άπληστες επιλογές.
- Ο **A*** επιτυγχάνει σημαντική μείωση βημάτων σε σχέση με τους **DFS** και **Best-First**, ειδικά σε προβλήματα μεγάλου μεγέθους.

Πίνακας 3: Διάγραμμα Μήκους Λύσης

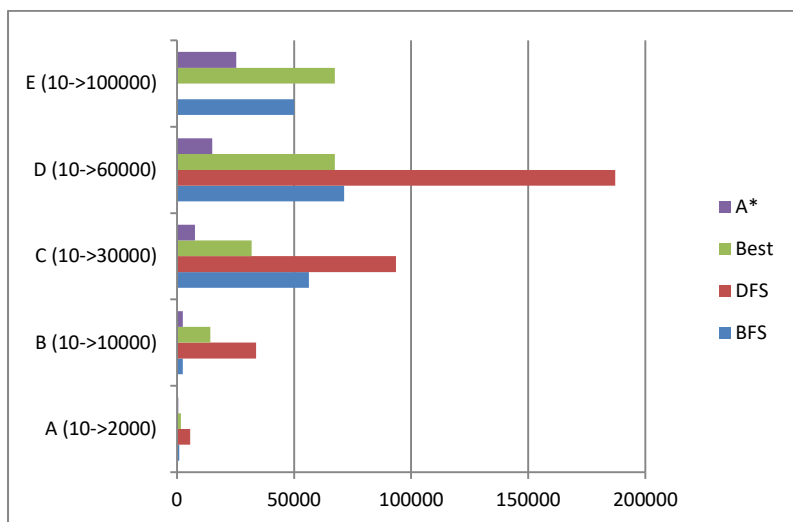


4.4. Σύγκριση Κόστους Λύσης

- Ο **BFS** παράγει συχνά πολύ ακριβές λύσεις.
- Ο **DFS** εμφανίζει τις λύσεις με το χειρότερο κόστος.
- Ο **Best-First** αποτυγχάνει σε μεγάλα προβλήματα.

- Ο **A*** παράγει σε όλα τα σενάρια λύσεις βέλτιστες ως προς το κόστος ή ισοδύναμες επιβεβαιώνοντας ότι η ευρετική συνάρτηση είναι παραδεκτή.

Πίνακας 4: Σύγκριση Κόστους Λύσης



4.5. Τελικά Συμπεράσματα και Συνολική Αξιολόγηση

Συνοψίζοντας τα ευρήματα της πειραματικής διαδικασίας παρατηρήσαμε ότι η επιλογή του κατάλληλου αλγορίθμου σχετίζεται άμεσα με τις απαιτήσεις του προβλήματος, δηλαδή αν ζητάει βελτιστοποίηση είτε ως προς το πλήθος βημάτων, είτε ως προς το κόστος είτε ως προς την ταχύτητα. Αναλυτικότερα:

1. **Αναζήτηση Σε Πλάτος:** Συμπεράναμε ότι ο BFS αλγόριθμος είναι αξιόπιστος μόνο για εύρεση λύσεων με βελτιστοποίηση ως προς τον αριθμό των βημάτων. Ωστόσο η τυφλή/μη-πληροφορημένη φύση του που αγνοεί τα κόστη των πράξεων τον καθιστά ακατάλληλο για προβλήματα με στόχο την βελτιστοποίηση του κόστους καθώς πολλές φορές επιλέγει μονοπάτια μόνο και μόνο επειδή είναι σύντομα.
2. **Αναζήτηση Σε Βάθος:** Συμπεράναμε πως ο DFS είναι από τους πιο ακατάλληλους, αν όχι ο πιο ακατάλληλος αλγόριθμος για το συγκεκριμένο πρόβλημα, από όλες τις οπτικές γωνίες τις βελτιστοποίησης. Η τάση του να εμβαθύνει απεριόριστα μας οδηγεί σε υπερβολική κατανάλωση πόρων με υψηλό κόστος εκτέλεσης και τεράστιους χρόνους εκτέλεσης. Το φαινόμενο λήξης του χρονικού ορίου στο Σενάριο(10->100.000) επιβεβαιώνει την αδυναμία του να διαχειριστεί μεγάλους χώρους καταστάσεων χωρίς όριο βάθους.
3. **Αναζήτηση Πρώτα Στο Καλύτερο:** Συμπεράναμε ότι ο Best-First (Greedy) λειτούργησε ως μια γρήγορη εναλλακτική λόγω της πληροφορημένης του φύσης, δίνοντας μας τις περισσότερες φορές γρήγορη λύση. Παρόλα αυτά, η απληστία του αλγορίθμου τον οδήγησε συχνά σε τοπικά βέλτιστα, αποτυγχάνοντας να μας δώσει με σιγουριά τη συνολικά φθηνότερη λύση, ειδικά σε πολύπλοκα σενάρια.
4. **A*:** Συμπεράναμε μέσω της συγκριτικής αξιολόγησης ότι ο A* ήταν τελικά ο καταλληλότερος αλγόριθμος για αυτό το πρόβλημα καθώς συνδυάζοντας το πραγματικό κόστος διαδρομής και την ευρετική εκτίμηση, κατάφερε να μας δίνει πάντα την λύση με το ελάχιστο δυνατό κόστος, διατηρώντας χαμηλούς χρόνους εκτέλεσης, ακόμη και σε μεγάλα προβλήματα όπου άλλοι κατέρρευσαν, εξερευνώντας πάντοτε σημαντικά λιγότερους κόμβους από τους άλλους αλγορίθμους.